HULT INTERNATIONAL BUSINESS SCHOOL

Algorithms & Data Structures

Course Syllabus | Professor Roger Hunt

First draft, version 2. Term, dates, times, room and office hours are left blank so this syllabus can be reused.

Course Information

Course code	[Course code TBD]
Credit hours	[Credit hours TBD]
Term	[____] (15 weeks)
Dates	[____]
Meeting days / times	[____] (2 classes per week, 30 classes in all)
Campus / room	[____] / [____]
Instructor	Professor Roger Hunt
Email	[Email TBD]
Office hours	[____]
Prerequisites	[Prerequisites TBD]
Course site / LMS	[LMS link TBD]

Course Description

A core computer science course in which you learn algorithms and data structures by solving problems. In every class you solve two LeetCode problems at the board, an easier one and a harder one, writing Python in chalk with no devices. For each solution you state its time complexity and auxiliary space in Big-O, and you estimate the total memory it uses. The problems start at Easy, reach Medium in the middle of the term, and become Hard by the end.

Each class builds on the ones before it. Over 30 classes you work through Big-O and Python memory, arrays and strings, hashing, two pointers and sliding windows, stacks, queues and monotonic structures, linked lists, recursion, bit manipulation, sorting and binary search, trees, heaps, graphs (search, topological sort, shortest paths), union-find, tries, backtracking, greedy algorithms, intervals and dynamic programming.

Skills developed: algorithmic thinking, pattern recognition, data structure selection, complexity analysis (time, space and memory), problem decomposition, planning before coding, and fluent Python.

The course uses Python only. In class, all work is done by hand: you write Python in chalk on the board and then on a paper answer sheet.

Learning Outcomes

By the end of the course, you will be able to:

1. Derive the Big-O time and auxiliary space complexity of iterative and recursive Python code, including amortized cost, and justify the result in writing and aloud.
2. Estimate the total memory a Python solution uses for a given input size, using the sizes of CPython objects (int, str, list, dict, set, deque, node objects, call frames).
3. Recognize which pattern a new problem belongs to and defend the choice by comparing complexity.
4. Write correct Python solutions by hand to Easy, Medium and Hard problems.
5. Implement dynamic arrays, linked lists, stacks, queues, hash tables, heaps, binary trees, tries, union-find and graph representations without a computer.
6. Apply and trace core algorithms: sorting, binary search, tree and graph traversal, topological sort, Dijkstra, backtracking, greedy methods, interval sweeps and dynamic programming.
7. Plan before coding: state a brute-force approach, the pattern, the steps, the complexity and the edge cases in writing.
8. Find correctness bugs and hidden inefficiencies in AI-generated code by reading and tracing it, and correct them.

How This Course Works

This course runs on preparation and practice. I do not lecture. You learn the pattern before class, and class time is for doing the work.

Before class: the take-home

- **A take-home for every class.** At the end of each class you receive the take-home for the next one. (Take-Home 1 is posted before the first class.) It names the pattern or type of LeetCode problem you will meet in class, gives the Runestone reading, lists 2 to 3 practice problems of that type, and asks for a short plan.
- **Spend at least 2 hours.** Read the section, study the pattern, and attempt the practice problems on LeetCode. Using AI to learn is allowed and expected.
- **Plan before code.** Bring a written plan to class for one practice problem: the problem in your own words, the brute-force idea and its Big-O, the pattern you chose and why, the steps in plain English, the expected time, space and memory, and two edge cases. The plan is on paper, and it is how you arrive prepared.

In class

- **No lecturing.** Class starts with the work.
- **Two problems every class.** Problem A is easier and Problem B is harder. You are told the pattern in the take-home, not the problems. You get the problems when class starts.
- **Chalk first, then paper.** Solve each problem at the board in Python, in chalk. Then copy your solution to the paper answer sheet.
- **For every solution, state three things:** its time complexity, its auxiliary space, and an estimate of the total memory it uses (see the table below).
- **Devices are locked away.** Phones and computers are locked away for the whole session. There is no coding on a computer in class.
- **You work individually.** Everyone works on their own.
- **Stuck? Ask for help, and show your attempts.** If you are stuck, ask me. When you ask, you must show at least 2 failed attempts, and you record them in the failed-attempts box on your answer sheet.
- **Finish early, go further.** If you finish both problems correctly, I will give you an extension problem. [Extension format TBD by instructor]
- **Excel, and you publish.** Students who excel are expected to contribute to, and receive coauthor credit on, a paper I submit to a journal. See Research & Coauthorship Track.

What you write for every board problem

Item	What to write
Python solution	A plain function in readable Python. Use only what Python gives you, plus collections (deque) and heapq where natural. No libraries beyond those.
Time complexity	Big-O in terms of the input size. Name your variables: n, m, k, V, E.
Auxiliary space	Big-O of the extra memory your solution allocates beyond the input. Anything you build and return counts.
Memory estimate	For the input size given on the sheet, in bytes: the memory of the input, the memory of your solution's auxiliary data, and their total. State your assumptions.
Failed attempts	If you ask for help, two failed attempts, written before you ask.

Python memory: reference sizes

Use these sizes for the memory estimate. They are CPython 3.13 on a 64-bit machine, measured with sys.getsizeof and tracemalloc. Treat results as estimates (our checks agree within about 25% to a factor of 2 on the large cases).

Object	Approximate size
int	28 bytes (the ints -5 to 256 are shared and cost nothing extra)
float	24 bytes
True, False, None	Shared objects: no extra cost
str	41 bytes + 1 byte per ASCII character (a 1-character string is shared)
list of n items	56 bytes + 8 bytes per slot, plus about 12% spare room if built with append. It holds pointers: each object it refers to is counted separately.
tuple of k items	40 bytes + 8 bytes per item (56 bytes for a pair, 64 for a triple)
dict	About 50 bytes per entry for large dicts (37 at 1,000 entries; 184 bytes for a tiny dict). Keys and values are counted separately.
set	About 42 bytes per element for large sets (33 at 1,000 entries). Elements are counted separately.
deque	About 8.3 bytes per element, plus about 600 bytes overhead
Node object (ListNode, TreeNode)	About 100 bytes, including its value and pointers
Recursive call	About 150 bytes per active call (the call stack)

How the difficulty ramps

Classes	Level of the two problems	Focus
1 to 8	Easy and Easy	Foundations: Big-O, memory, arrays, hashing, pointers, windows, stacks, queues
9 to 19	Easy and Medium	Monotonic structures, linked lists, recursion, bits, sorting, search, trees, heaps, graphs
20 to 22	Medium and Medium	Topological sort, shortest paths, union-find
23 to 26	Medium and Hard	Tries, backtracking, greedy, intervals
27 to 28	Medium and Medium	Dynamic programming begins again at Medium: 1D, then sequences
29 to 30	Medium or Hard, then Hard	Two-dimensional DP, then the capstone with no topic label

Class Schedule

30 classes: 2 per week for 15 weeks [adjust if the term differs]. Two real LeetCode problems per class; number and difficulty as listed on leetcode.com.

Wk	Class	Topic / pattern	Level	Problem A (easier)	Problem B (harder)
1	1	Big-O and Python memory	Easy / Easy	#1480 Running Sum of 1d Array	#217 Contains Duplicate
	2	Arrays and in-place updates	Easy / Easy	#283 Move Zeroes	#88 Merge Sorted Array
2	3	Strings and prefix sums	Easy / Easy	#14 Longest Common Prefix	#303 Range Sum Query - Immutable
	4	Hash sets and hash maps	Easy / Easy	#242 Valid Anagram	#1 Two Sum
3	5	Two pointers	Easy / Easy	#125 Valid Palindrome	#977 Squares of a Sorted Array
	6	Sliding window	Easy / Easy	#643 Maximum Average Subarray I	#121 Best Time to Buy and Sell Stock
4	7	Stacks	Easy / Easy	#20 Valid Parentheses	#844 Backspace String Compare
	8	Queues and deques	Easy / Easy	#933 Number of Recent Calls	#232 Implement Queue using Stacks
5	9	Monotonic stack and deque	Easy / Medium	#496 Next Greater Element I	#739 Daily Temperatures
	10	Linked lists	Easy / Medium	#206 Reverse Linked List	#19 Remove Nth Node From End of List
6	11	Recursion	Easy / Medium	#509 Fibonacci Number	#50 Pow(x, n)
	12	Bit manipulation	Easy / Medium	#136 Single Number	#260 Single Number III
7	13	Sorting	Easy / Medium	#1331 Rank Transform of an Array	#912 Sort an Array
	14	Binary search	Easy / Medium	#704 Binary Search	#34 Find First and Last Position of Element in Sorted Array
8	15	Trees: traversal	Easy / Medium	#104 Maximum Depth of Binary Tree	#102 Binary Tree Level Order Traversal
	16	Binary search trees	Easy / Medium	#700 Search in a Binary Search Tree	#98 Validate Binary Search Tree
9	17	Heaps and priority queues	Easy / Medium	#703 Kth Largest Element in a Stream	#347 Top K Frequent Elements
	18	Graphs and depth-first search	Easy / Medium	#733 Flood Fill	#200 Number of Islands
10	19	Breadth-first search	Easy / Medium	#1971 Find if Path Exists in Graph	#994 Rotting Oranges
	20	Topological sort	Medium / Medium	#207 Course Schedule	#210 Course Schedule II
11	21	Shortest paths: Dijkstra	Medium / Medium	#743 Network Delay Time	#1631 Path With Minimum Effort
	22	Union-find	Medium / Medium	#547 Number of Provinces	#684 Redundant Connection
12	23	Tries	Medium / Hard	#208 Implement Trie (Prefix Tree)	#212 Word Search II
	24	Backtracking	Medium / Hard	#78 Subsets	#51 N-Queens
13	25	Greedy algorithms	Medium / Hard	#55 Jump Game	#135 Candy
	26	Intervals and sweep line	Medium / Hard	#56 Merge Intervals	#218 The Skyline Problem
14	27	Dynamic programming: 1D	Medium / Medium	#198 House Robber	#322 Coin Change
	28	Dynamic programming: sequences	Medium / Medium	#300 Longest Increasing Subsequence	#1143 Longest Common Subsequence
15	29	Dynamic programming: 2D tables	Medium / Hard	#72 Edit Distance	#10 Regular Expression Matching
	30	Capstone: choosing the pattern	Hard / Hard	#127 Word Ladder	#295 Find Median from Data Stream

Problems are given in class. The take-home for each class tells you only the pattern and gives different practice problems of the same type.

Grading

The rule

Attend every class and put in solid effort, as judged by me, and you earn an A.

Solid effort means this: if you are stuck, you ask for help, and when you ask for help you show at least 2 failed attempts. Separately, you are expected to arrive prepared, with your take-home plan written. Your grade comes from attendance and effort under this rule.

Extra credit and covering a missed class

Correct answers earn extra credit points. A correct answer has a working solution and the right time, space and memory analysis. You can use those points to cover a missed class, so that an absence does not count against you. Points come from correct answers on the board problems and on extension problems. The specific values are set by the instructor:

Points per correct board problem	[TBD by instructor]
Points per correct extension problem	[TBD by instructor]
Points needed to cover one missed class	[TBD by instructor]
Maximum absences that can be covered	[TBD by instructor]
Effect of an uncovered absence on the grade	[TBD by instructor]
Late arrival / early departure	[TBD by instructor]

AI Use Policy

AI use is allowed and expected in this course, outside class. I expect you to use AI to learn. In class, devices are locked away, so the work you do in the room is yours alone.

What is expected

- **Prep.** Use AI to explain the pattern, walk through examples, quiz you, and generate more practice problems before class.
- **Practice outside class.** Write your plan and your attempt first. Then use AI to check your plan and your solution, and to find where your reasoning broke.
- **Verification.** Treat AI output as a draft. AI tools regularly produce code that runs but is wrong or slower than claimed. Check complexity claims by hand.
- **Transparency.** Be ready to show which tools you used and how, if I ask.

The one requirement

You must be able to explain and defend your work in class. If I ask why an answer is correct, how it works, or what you would change under different conditions, you need to answer on the spot. An answer you cannot explain does not count as a correct answer for extra credit.

Research & Coauthorship Track

Students who excel in this course are expected to contribute to a paper I submit to a peer-reviewed journal, and they receive coauthor credit for that contribution.

- **How you get there.** Consistently strong work, especially on the harder problems and extension problems, is the signal. I will invite students directly. [Selection criteria and timing TBD by instructor]
- **What contributing looks like.** Designing or running experiments and benchmarks, implementing and testing algorithms, analyzing results, reviewing literature, and drafting sections of the paper.
- **Paper topic and target journal.** [TBD by instructor]
- **Authorship.** Contribution expectations and author order will be agreed in writing before work begins. [Authorship agreement details TBD]
- **Timeline.** Work may continue after the semester ends, through submission and revision. [Timeline TBD]

Tools

- Language: Python 3, only.
- For out-of-class prep: any AI assistant, a free LeetCode account for the practice problems, and Python's timeit and sys.getsizeof for checking your own predictions. [Licensing / access arrangements TBD]
- In class: chalk, the board, pen, and the paper answer sheets I provide. No phones or computers.

Materials

Required (free)

- Brad Miller and David Ranum, Problem Solving with Algorithms and Data Structures using Python, 3rd edition (free on Runestone Academy). Primary text. Every take-home names the sections to read. https://runestone.academy/ns/books/published/pythonds3/index.html
- Pat Morin, Open Data Structures (free online). Use it for the data structure chapters. https://opendatastructures.org/

- Jeff Erickson, Algorithms (free online). Use it for recursion, graphs, greedy and dynamic programming. https://jeffe.cs.illinois.edu/teaching/algorithms/
- MIT OpenCourseWare 6.006, Introduction to Algorithms (Spring 2020): lecture videos and notes (free). Taught in Python.
- LeetCode (free account): the practice problems in the take-homes and the problems in class.
- Official documentation: the Python documentation (built-in types, collections, heapq, bisect, sys.getsizeof).

Recommended

- Cormen, Leiserson, Rivest, and Stein, Introduction to Algorithms (CLRS), 4th edition. Standard reference.
- VisuAlgo, free interactive visualizations of data structures and algorithms. https://visualgo.net/en

Runestone does not cover every pattern in this course (for example sliding windows, union-find, tries and intervals). Where a take-home says so, it points you to another source.

Institutional Policies

The policies below are governed by Hult's official documents. Where this syllabus and an official Hult policy differ, the Hult policy applies.

Academic integrity

You are expected to do honest work and represent it accurately. Using AI as described in the AI Use Policy is permitted and expected; presenting work you cannot explain, or misrepresenting who did the work, is not. See Hult's official academic integrity policy [link TBD].

Accessibility and accommodations

If you need an accommodation, contact the appropriate Hult office as early as possible and let me know once it is arranged. See Hult's official accessibility and accommodations policy [link TBD].

Attendance

Attendance is central to this course because all graded work happens in class. Absences are also subject to Hult's official attendance policy [link TBD]. How absences affect your grade in this course is described under Grading.

Changes to this syllabus

I may adjust topics or pacing during the term. Changes will be announced in class and on [LMS TBD].